```
import numpy as np
import pandas as pd
```

```
dataset = pd.read_csv('/content/Bank_data.csv')
display(dataset)
```

| | Customer_ID | Age | Gender | Marital_Status | Employment_Type | Annual_I |
|---|---|---|---|---|---|---|
| 0 | CUST100000 | 56.0 | NaN | Marrid | Unemployed | |
| 1 | CUST100001 | 69.0 | Mal | Marrid | Salaried | |
| 2 | CUST100002 | 46.0 | Other | Singl | Salaried | |
| 3 | CUST100003 | 32.0 | NaN | Widowed | Unemployed | 1,: |
| 4 | CUST100004 | 60.0 | Female | Single | Self-Employed | 2 |
| ... | ... | ... | ... | ... | ... | |
| 15745 | CUST113584 | 40.0 | Male | Married | Salaried | ₹ |
| 15746 | CUST112144 | 49.0 | Female | Married | Self-Employed | ₹ |
| 15747 | CUST114397 | 54.0 | NaN | Singl | Salaried | |
| 15748 | CUST108787 | 30.0 | Male | Divorced | Self-Employed | |
| 15749 | CUST114191 | 29.0 | Femlae | Divorced | Salaried | |

15750 rows × 61 columns

```
pd.options.display.max_columns = None
pd.options.display.max_rows = None
```

```
dataset.shape
```

```
(15750, 61)
```

```
dataset.head(20)
```

| | Customer_ID | Age | Gender | Marital_Status | Employment_Type | Annual_Inc |
|---|---|---|---|---|---|---|
| 0 | CUST100000 | 56.0 | NaN | Marrid | Unemployed | 50 |
| 1 | CUST100001 | 69.0 | Mal | Marrid | Salaried | 50 |
| 2 | CUST100002 | 46.0 | Other | Singl | Salaried | I |
| 3 | CUST100003 | 32.0 | NaN | Widowed | Unemployed | 1,20, |
| 4 | CUST100004 | 60.0 | Female | Single | Self-Employed | 200 |
| 5 | CUST100005 | 25.0 | Other | Married | Salaried | 1 |
| 6 | CUST100006 | 38.0 | Mal | Widowed | NaN | 200 |
| 7 | CUST100007 | 56.0 | NaN | Single | Jobless | I |
| 8 | CUST100008 | 36.0 | Male | Single | NaN | 1 |
| 9 | CUST100009 | 40.0 | NaN | Widowed | Salaried | ₹80 |
| 10 | CUST100010 | 28.0 | NaN | Divorced | Unemployed | ₹80 |
| 11 | CUST100011 | 28.0 | Femlae | Widowed | Self-Employed | 200 |
| 12 | CUST100012 | 41.0 | NaN | Marrid | Salaried | 1,20, |
| 13 | CUST100013 | 150.0 | Mal | NaN | Unemployed | 50 |
| 14 | CUST100014 | 53.0 | Male | Singl | Jobless | ₹80 |
| 15 | CUST100015 | 57.0 | Mal | Marrid | Retired | I |
| 16 | CUST100016 | 41.0 | Female | Single | Jobless | 1,20, |
| 17 | CUST100017 | 20.0 | Other | Single | Jobless | 1,20, |
| 18 | CUST100018 | 39.0 | NaN | Married | Self-Employed | 1 |
| 19 | CUST100019 | 150.0 | Femlae | Single | Salaried | ₹80 |

```
dataset.describe()
```

| | Age | Current_Residence_Years | SK_ID_CURR | DAYS_BIRTH | DAY |
|---|---|---|---|---|---|
| count | 15449.000000 | 15750.000000 | 15750.000000 | 15750.000000 | 1 |
| mean | 44.264742 | 14.525333 | 551982.580063 | -16035.809587 | - |
| std | 21.410577 | 8.680826 | 259156.732876 | 5191.822934 | |
| min | -5.000000 | 0.000000 | 100077.000000 | -24999.000000 | -1 |
| 25% | 30.000000 | 7.000000 | 327897.000000 | -20551.750000 | -1 |
| 50% | 43.000000 | 15.000000 | 554029.000000 | -16071.000000 | - |
| 75% | 57.000000 | 22.000000 | 775903.250000 | -11526.000000 | - |
| max | 150.000000 | 29.000000 | 999986.000000 | -7002.000000 | |

```
dataset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15750 entries, 0 to 15749
Data columns (total 61 columns):
 #   Column                       Non-Null Count  Dtype
---  ------                       --------------  -----
 0   Customer_ID                  15750 non-null  object
 1   Age                          15449 non-null  float64
 2   Gender                       13103 non-null  object
 3   Marital_Status               13463 non-null  object
 4   Employment_Type              13064 non-null  object
 5   Annual_Income                13089 non-null  object
 6   Loan_Amount                  12642 non-null  object
 7   Loan_Term_Months             13555 non-null  object
 8   Credit_Score                 13109 non-null  object
 9   Number_of_Dependents         13456 non-null  object
 10  Past_Defaults                12540 non-null  object
 11  Loan_Purpose                 13498 non-null  object
 12  Current_Residence_Years      15750 non-null  int64
 13  Default                      13530 non-null  object
 14  SK_ID_CURR                   15750 non-null  int64
 15  TARGET                       11894 non-null  object
 16  NAME_CONTRACT_TYPE           15750 non-null  object
 17  FLAG_OWN_CAR                 12594 non-null  object
 18  FLAG_OWN_REALTY              12605 non-null  object
 19  CNT_CHILDREN                 13177 non-null  object
 20  AMT_INCOME_TOTAL             12547 non-null  object
 21  AMT_CREDIT                   12681 non-null  object
 22  AMT_ANNUITY                  12601 non-null  object
 23  AMT_GOODS_PRICE              11766 non-null  object
 24  NAME_INCOME_TYPE             13125 non-null  object
 25  NAME_EDUCATION_TYPE          13138 non-null  object
 26  NAME_FAMILY_STATUS           12533 non-null  object
 27  NAME_HOUSING_TYPE            12604 non-null  object
 28  DAYS_BIRTH                   15750 non-null  int64
```

```
 29  DAYS_EMPLOYED                15750 non-null  int64
 30  OCCUPATION_TYPE              13440 non-null  object
 31  CNT_FAM_MEMBERS              15750 non-null  int64
 32  REGION_RATING_CLIENT         11899 non-null  float64
 33  WEEKDAY_APPR_PROCESS_START   13085 non-null  object
 34  HOUR_APPR_PROCESS_START      13758 non-null  object
 35  ORGANIZATION_TYPE            13131 non-null  object
 36  EXT_SOURCE_1                 9517 non-null   float64
 37  EXT_SOURCE_2                 12654 non-null  object
 38  EXT_SOURCE_3                 12645 non-null  object
 39  Unwanted_Column_1            11021 non-null  object
 40  Unwanted_Column_2            11074 non-null  object
 41  Unwanted_Column_3            11081 non-null  object
 42  Unwanted_Column_4            11017 non-null  object
 43  Unwanted_Column_5            10981 non-null  object
 44  Unwanted_Column_6            10982 non-null  object
 45  Unwanted_Column_7            11020 non-null  object
 46  Unwanted_Column_8            11006 non-null  object
 47  Unwanted_Column_9            10958 non-null  object
 48  Unwanted_Column_10           10954 non-null  object
 49  Unwanted_Column_11           10987 non-null  object
 50  Unwanted_Column_12           10998 non-null  object
 51  Unwanted_Column_13           11022 non-null  object
 52  Unwanted_Column_14           11021 non-null  object
```

```
dataset.tail(5)
```

| | Customer_ID | Age | Gender | Marital_Status | Employment_Type | Annual_I |
|---|---|---|---|---|---|---|
| **15745** | CUST113584 | 40.0 | Male | Married | Salaried | ₹ |
| **15746** | CUST112144 | 49.0 | Female | Married | Self-Employed | ₹ |
| **15747** | CUST114397 | 54.0 | NaN | Singl | Salaried | |
| **15748** | CUST108787 | 30.0 | Male | Divorced | Self-Employed | |
| **15749** | CUST114191 | 29.0 | Femlae | Divorced | Salaried | |

```
(dataset.isnull().sum()/len(dataset))*100
```

```
dataset.duplicated().sum()
```

```
np.int64(750)
```

```
dataset.head(5)
```

| | Customer_ID | Age | Gender | Marital_Status | Employment_Type | Annual_Incom |
|---|---|---|---|---|---|---|
| 0 | CUST100000 | 56.0 | NaN | Marrid | Unemployed | 5000 |
| 1 | CUST100001 | 69.0 | Mal | Marrid | Salaried | 5000 |
| 2 | CUST100002 | 46.0 | Other | Singl | Salaried | Na |
| 3 | CUST100003 | 32.0 | NaN | Widowed | Unemployed | 1,20,00 |
| 4 | CUST100004 | 60.0 | Female | Single | Self-Employed | 20000 |

```
dataset.columns = dataset.columns.str.lower().str.strip()
```

```
dataset['age'] = dataset['age'].fillna(dataset['age'].median()).astype(int)
```

```
dataset['age'].isna().sum()
```

```
np.int64(0)
```

```
dataset.head(5)
```

| | customer_id | age | gender | marital_status | employment_type | annual_incom |
|---|---|---|---|---|---|---|
| 0 | CUST100000 | 56 | NaN | Marrid | Unemployed | 5000 |
| 1 | CUST100001 | 69 | Mal | Marrid | Salaried | 5000 |
| 2 | CUST100002 | 46 | Other | Singl | Salaried | Nal |
| 3 | CUST100003 | 32 | NaN | Widowed | Unemployed | 1,20,00 |
| 4 | CUST100004 | 60 | Female | Single | Self-Employed | 20000 |

```
dataset['gender'] = dataset['gender'].str.strip().str.lower()
```

```
dataset['gender'] = dataset['gender'].replace({
    'male': 'male',
    'mal': 'male',
```

```
    'female': 'female',
    'fem': 'female',
    'femlae':'female',
    'other': 'other'
})
```

```
dataset['age'].unique()
```

```
array([ 56,  69,  46,  32,  60,  25,  38,  36,  40,  28,  41, 150,  53,
        57,  20,  39,  19,  61,  47,  55,  50,  29,  42,  66,  44,  59,
        45,  33,  64,  68,  43,  54,  24,  26,  35,  21,  31,  67,  37,
        52,  34,  23,  -5,  51,  27,  48,  65,  62,  58,  18,  22,  30,
        49,  63])
```

```
dataset.head(5)
```

| | customer_id | age | gender | marital_status | employment_type | annual_incom |
|---|---|---|---|---|---|---|
| 0 | CUST100000 | 56 | NaN | Marrid | Unemployed | 5000 |
| 1 | CUST100001 | 69 | male | Marrid | Salaried | 5000 |
| 2 | CUST100002 | 46 | other | Singl | Salaried | Nal |
| 3 | CUST100003 | 32 | NaN | Widowed | Unemployed | 1,20,00 |
| 4 | CUST100004 | 60 | female | Single | Self-Employed | 20000 |

```
dataset['gender'].value_counts()
```

| gender | count |
|---|---|
| female | 5284 |
| male | 5278 |
| other | 2541 |

dtype: int64

```
dataset['gender'] = dataset['gender'].fillna(dataset['gender'].mode()[0])
```

```
dataset['marital_status'] = dataset['marital_status'].str.lower().str.strip()
```

```
dataset['marital_status'].value_counts()
```

| marital_status | count |
|---|---|
| singl | 2417 |
| widowed | 2262 |
| divorced | 2255 |
| marrid | 2206 |
| married | 2170 |
| single | 2153 |

dtype: int64

```
dataset['marital_status'] = dataset['marital_status'].replace({
    'singl':'single',
    'marrid':'married'
})
```

```
dataset['marital_status'].isnull().sum()
```

```
np.int64(2287)
```

```
dataset['marital_status'] = dataset['marital_status'].fillna('unknown')
```

```
dataset.head(5)
```

| | customer_id | age | gender | marital_status | employment_type | annual_incom |
|---|---|---|---|---|---|---|
| 0 | CUST100000 | 56 | female | married | Unemployed | 5000 |
| 1 | CUST100001 | 69 | male | married | Salaried | 5000 |
| 2 | CUST100002 | 46 | other | single | Salaried | Nal |
| 3 | CUST100003 | 32 | female | widowed | Unemployed | 1,20,00 |
| 4 | CUST100004 | 60 | female | single | Self-Employed | 20000 |

```
dataset['employment_type'].value_counts()
```

| employment_type | count |
|---|---|
| Self-Employed | 2733 |
| Retired | 2696 |
| Jobless | 2547 |
| Salaried | 2545 |
| Unemployed | 2543 |

dtype: int64

```
dataset['employment_type'] = dataset['employment_type'].str.lower().str.strip(
```

```
dataset['employment_type'].isna().sum()
```

```
np.int64(2686)
```

```
dataset['employment_type'] = dataset['employment_type'].fillna(dataset['employ
```

```
dataset['employment_type'].value_counts()
```

| employment_type | count |
|---|---|
| self-employed | 5419 |
| retired | 2696 |
| jobless | 2547 |
| salaried | 2545 |
| unemployed | 2543 |

dtype: int64

```
dataset.head(5)
```

| | customer_id | age | gender | marital_status | employment_type | annual_incom |
|---|---|---|---|---|---|---|
| 0 | CUST100000 | 56 | female | married | unemployed | 5000 |
| 1 | CUST100001 | 69 | male | married | salaried | 5000 |
| 2 | CUST100002 | 46 | other | single | salaried | Nal |
| 3 | CUST100003 | 32 | female | widowed | unemployed | 1,20,00 |
| 4 | CUST100004 | 60 | female | single | self-employed | 20000 |

```
dataset['annual_income'].value_counts()
```

| annual_income | count |
|---|---|
| 1,20,000 | 2678 |
| 50000 | 2649 |
| 100k | 2610 |
| ₹80000 | 2585 |
| 200000 | 2567 |

dtype: int64

```
import numpy as np

def clean_income(x):
    if pd.isna(x):
        return np.nan
    x = str(x).lower().replace(',', '').strip()

    # Handle 'k' or 'lakh' suffix
    if 'k' in x:
        return float(x.replace('k', '')) * 1000
    elif 'lakh' in x:
        return float(x.replace('lakh', '').strip()) * 100000
    else:
        try:
            return float(x)
        except:
            return np.nan

dataset['annual_income'] = dataset['annual_income'].apply(clean_income)
```

```
dataset['annual_income'] = dataset['annual_income'].replace('[\$,]', '', regex=
```

```
<>:1: SyntaxWarning: invalid escape sequence '\$'
<>:1: SyntaxWarning: invalid escape sequence '\$'
/tmp/ipython-input-1452909548.py:1: SyntaxWarning: invalid escape sequenc
  dataset['annual_income'] = dataset['annual_income'].replace('[\$,]', ''
```

```
dataset['annual_income'].describe()
```

| | annual_income |
|---|---|
| count | 10504.000000 |
| mean | 116927.837014 |
| std | 53758.801073 |
| min | 50000.000000 |
| 25% | 50000.000000 |
| 50% | 100000.000000 |
| 75% | 120000.000000 |
| max | 200000.000000 |

dtype: float64

```
dataset['annual_income'] = dataset['annual_income'].fillna(dataset['annual_inc
```

```
dataset['annual_income'].isna().sum()
```

```
np.int64(0)
```

```
dataset['annual_income'] = dataset['annual_income'].astype(int)
```

```
dataset['annual_income'].shape
```

```
(15750,)
```

```
dataset.head(5)
```

| | customer_id | age | gender | marital_status | employment_type | annual_incom |
|---|---|---|---|---|---|---|
| 0 | CUST100000 | 56 | female | married | unemployed | 5000 |
| 1 | CUST100001 | 69 | male | married | salaried | 5000 |
| 2 | CUST100002 | 46 | other | single | salaried | 10000 |
| 3 | CUST100003 | 32 | female | widowed | unemployed | 12000 |
| 4 | CUST100004 | 60 | female | single | self-employed | 20000 |

```
dataset['loan_amount'].isna().sum()
```

```
np.int64(3108)
```

```
dataset['loan_amount'].value_counts()
```

| loan_amount | count |
|---|---|
| 150000 | 3194 |
| 10000 | 3170 |
| ₹200000 | 3152 |
| 300k | 3126 |

dtype: int64

```
import numpy as np
import pandas as pd

def clean_loan_amount(x):
    if pd.isna(x):
        return np.nan

    # convert to lowercase string and remove symbols/spaces
    x = str(x).lower().replace(',', '').replace('₹', '').replace('$', '').strip

    # handle k/lakh suffixes
    if 'k' in x:
        num = x.replace('k', '').strip()
        return float(num) * 1000 if num.replace('.', '', 1).isdigit() else np.n
    elif 'lakh' in x:
        num = x.replace('lakh', '').strip()
        return float(num) * 100000 if num.replace('.', '', 1).isdigit() else np

    # handle plain numeric strings
    if x.replace('.', '', 1).isdigit():
        return float(x)
```

```
    # invalid or text-like entries
    return np.nan


# Apply cleaning function
dataset['loan_amount'] = dataset['loan_amount'].apply(clean_loan_amount)

# Optional: Fill missing values with median for project simplicity
dataset['loan_amount'] = dataset['loan_amount'].fillna(dataset['loan_amount'].
```

```
dataset['loan_amount'].isna().sum()
```

```
np.int64(0)
```

```
dataset['loan_amount'] = dataset['loan_amount'].astype(int)
```

```
dataset.head(5)
```

| | customer_id | age | gender | marital_status | employment_type | annual_incom |
|---|---|---|---|---|---|---|
| 0 | CUST100000 | 56 | female | married | unemployed | 5000 |
| 1 | CUST100001 | 69 | male | married | salaried | 5000 |
| 2 | CUST100002 | 46 | other | single | salaried | 10000 |
| 3 | CUST100003 | 32 | female | widowed | unemployed | 12000 |
| 4 | CUST100004 | 60 | female | single | self-employed | 20000 |

```
dataset.columns
```

```
Index(['customer_id', 'age', 'gender', 'marital_status',
'employment_type',
       'annual_income', 'loan_amount', 'loan_term_months',
'credit_score',
       'number_of_dependents', 'past_defaults', 'loan_purpose',
       'current_residence_years', 'default', 'sk_id_curr', 'target',
       'name_contract_type', 'flag_own_car', 'flag_own_realty',
'cnt_children',
       'amt_income_total', 'amt_credit', 'amt_annuity',
'amt_goods_price',
       'name_income_type', 'name_education_type', 'name_family_status',
       'name_housing_type', 'days_birth', 'days_employed',
'occupation_type',
       'cnt_fam_members', 'region_rating_client',
'weekday_appr_process_start',
       'hour_appr_process_start', 'organization_type', 'ext_source_1',
       'ext_source_2', 'ext_source_3', 'unwanted_column_1',
       'unwanted_column_2', 'unwanted_column_3', 'unwanted_column_4',
```

```
       'unwanted_column_5', 'unwanted_column_6', 'unwanted_column_7',
       'unwanted_column_8', 'unwanted_column_9', 'unwanted_column_10',
       'unwanted_column_11', 'unwanted_column_12', 'unwanted_column_13',
       'unwanted_column_14', 'unwanted_column_15', 'unwanted_column_16',
       'unwanted_column_17', 'unwanted_column_18', 'unwanted_column_19',
       'unwanted_column_20', 'unwanted_column_21',
'unwanted_column_22'],
      dtype='object')
```

```
# Columns to drop
drop_cols = [
    'unwanted_column_1', 'unwanted_column_2', 'unwanted_column_3',
    'unwanted_column_4', 'unwanted_column_5', 'unwanted_column_6',
    'unwanted_column_7', 'unwanted_column_8', 'unwanted_column_9',
    'unwanted_column_10', 'unwanted_column_11', 'unwanted_column_12',
    'unwanted_column_13', 'unwanted_column_14', 'unwanted_column_15',
    'unwanted_column_16', 'unwanted_column_17', 'unwanted_column_18',
    'unwanted_column_19', 'unwanted_column_20', 'unwanted_column_21',
    'unwanted_column_22'

]

# Drop them safely (ignore errors if some columns are missing)
dataset.drop(columns=drop_cols, inplace=True, errors='ignore')

# Check remaining columns
print("Remaining columns:\n", dataset.columns.tolist())
```

```
Remaining columns:
 ['customer_id', 'age', 'gender', 'marital_status', 'employment_type', 'a
```

```
dataset.head(5)
```

| | customer_id | age | gender | marital_status | employment_type | annual_incom |
|---|---|---|---|---|---|---|
| 0 | CUST100000 | 56 | female | married | unemployed | 5000 |
| 1 | CUST100001 | 69 | male | married | salaried | 5000 |
| 2 | CUST100002 | 46 | other | single | salaried | 10000 |
| 3 | CUST100003 | 32 | female | widowed | unemployed | 12000 |
| 4 | CUST100004 | 60 | female | single | self-employed | 20000 |

```
dataset['age'].value_counts()
```

```
dataset['age'] = dataset['age'].apply(
    lambda x: 25 if x < 18 else (32 if x > 100 else x)
)
```

```
dataset['gender'].value_counts()
```

| gender | count |
|---|---|
| female | 7931 |
| male | 5278 |
| other | 2541 |

dtype: int64

```
dataset['loan_term_months'].dtype
```

```
dtype('O')
```

```
dataset['loan_term_months'].value_counts()
```

| loan_term_months | count |
|---|---|
| 24 | 2310 |
| 60 | 2300 |
| 12 | 2261 |
| 12 months | 2241 |
| 48 | 2222 |
| 36 | 2221 |

dtype: int64

```
dataset['loan_term_months'] = dataset['loan_term_months'].astype(str).str.repla
dataset['loan_term_months'] = pd.to_numeric(dataset['loan_term_months'], errors
```

```
median_term = dataset['loan_term_months'].median()
dataset['loan_term_months'].fillna(median_term, inplace=True)
```

```
/tmp/ipython-input-3573966888.py:2: FutureWarning: A value is trying to b
The behavior will change in pandas 3.0. This inplace method will never wo

For example, when doing 'df[col].method(value, inplace=True)', try using


  dataset['loan_term_months'].fillna(median_term, inplace=True)
```

```
dataset['loan_term_months'].value_counts()
```

| loan_term_months | count |
|---|---|
| 24.0 | 4505 |
| 12.0 | 4502 |
| 60.0 | 2300 |
| 48.0 | 2222 |
| 36.0 | 2221 |

dtype: int64

```
dataset.head(5)
```

| | customer_id | age | gender | marital_status | employment_type | annual_incom |
|---|---|---|---|---|---|---|
| 0 | CUST100000 | 56 | female | married | unemployed | 5000 |
| 1 | CUST100001 | 69 | male | married | salaried | 5000 |
| 2 | CUST100002 | 46 | other | single | salaried | 10000 |
| 3 | CUST100003 | 32 | female | widowed | unemployed | 12000 |
| 4 | CUST100004 | 60 | female | single | self-employed | 20000 |

```
dataset['credit_score'].isna().sum()
```

```
np.int64(2641)
```

```
dataset['credit_score'] = pd.to_numeric(dataset['credit_score'], errors='coerc
dataset['credit_score'].fillna(dataset['credit_score'].median(), inplace=True)
dataset['credit_score'] = dataset['credit_score'].astype(int)
```

```
/tmp/ipython-input-3435825841.py:2: FutureWarning: A value is trying to b
The behavior will change in pandas 3.0. This inplace method will never wo

For example, when doing 'df[col].method(value, inplace=True)', try using


  dataset['credit_score'].fillna(dataset['credit_score'].median(), inplac
```

```
dataset['credit_score'].value_counts()
```

| credit_score | count |
|---|---|
| 450 | 7907 |
| 300 | 2698 |
| 700 | 2597 |
| 900 | 2548 |

dtype: int64

```
dataset['credit_score'].isna().sum()
```

```
np.int64(0)
```

```
dataset['number_of_dependents'].value_counts()
```

| number_of_dependents | count |
|---|---|
| 4 | 2295 |
| 2 | 2259 |
| 1 | 2255 |
| Unknown | 2244 |
| 3 | 2243 |
| 0 | 2160 |

dtype: int64

```
dataset['number_of_dependents'] = pd.to_numeric(dataset['number_of_dependents']
dataset.loc[dataset['number_of_dependents'] < 0, 'number_of_dependents'] = np.n
dataset['number_of_dependents'].fillna(dataset['number_of_dependents'].median(
```

```
dataset['number_of_dependents'] = dataset['number_of_dependents'].astype(int)
```

```
/tmp/ipython-input-3121816883.py:3: FutureWarning: A value is trying to b
The behavior will change in pandas 3.0. This inplace method will never wo

For example, when doing 'df[col].method(value, inplace=True)', try using


  dataset['number_of_dependents'].fillna(dataset['number_of_dependents'].
```

```
dataset['past_defaults'] = pd.to_numeric(dataset['past_defaults'], errors='coe
```

```
dataset.head(5)
```

| | customer_id | age | gender | marital_status | employment_type | annual_incom |
|---|---|---|---|---|---|---|
| 0 | CUST100000 | 56 | female | married | unemployed | 5000 |
| 1 | CUST100001 | 69 | male | married | salaried | 5000 |
| 2 | CUST100002 | 46 | other | single | salaried | 10000 |
| 3 | CUST100003 | 32 | female | widowed | unemployed | 12000 |
| 4 | CUST100004 | 60 | female | single | self-employed | 20000 |

```
dataset['loan_purpose'].value_counts()
```

| loan_purpose | count |
|---|---|
| Home | 2368 |
| Education | 2256 |
| Car | 2231 |
| Business | 2226 |
| Shopping | 2215 |
| Personal | 2202 |

dtype: int64

```
dataset['loan_purpose'].fillna(dataset['loan_purpose'].mode()[0], inplace=True
```

```
/tmp/ipython-input-753519944.py:1: FutureWarning: A value is trying to be
The behavior will change in pandas 3.0. This inplace method will never wo

For example, when doing 'df[col].method(value, inplace=True)', try using


  dataset['loan_purpose'].fillna(dataset['loan_purpose'].mode()[0], inpla
```

```
dataset['default'].value_counts()
```

| default | count |
|---|---|
| Yes | 2297 |
| No | 2295 |
| YES | 2280 |
| N | 2276 |
| Y | 2245 |
| NO | 2137 |

dtype: int64

```
dataset['default'].isna().sum()
```

```
np.int64(2220)
```

```
dataset['default'] = dataset['default'].astype(str).str.lower().str.strip()
dataset['default'] = dataset['default'].replace({'y': 'yes', 'n': 'no', 'no':
```

```
dataset['default'].unique()
```

```
array(['yes', 'no', 'nan'], dtype=object)
```

```
dataset['default'].unique()
```

```
array(['yes', 'no', 'nan'], dtype=object)
```

```
dataset['default'] = dataset['default'].replace({'yes': 1, 'no': 0})
```

```
dataset['default'] = pd.to_numeric(dataset['default'], errors='coerce')
```

```
dataset['default'] = dataset['default'].fillna(0).astype(int)
```

```
dataset['default'].value_counts()
```

| default | count |
|---|---|
| 0 | 8928 |
| 1 | 6822 |

dtype: int64

```
dataset.drop(['sk_id_curr', 'target'], axis=1, inplace=True)
```

```
dataset['name_contract_type'].value_counts()
```

| name_contract_type | count |
|---|---|
| CL | 3990 |
| Revolving loans | 3931 |
| RL | 3916 |
| Cash loans | 3913 |

dtype: int64

```
dataset['name_contract_type'] = dataset['name_contract_type'].str.lower().repla
```

```
dataset['name_contract_type'] = dataset['name_contract_type'].str.lower().repla
```

```
dataset['name_contract_type'].isna().sum()
```

```
np.int64(0)
```

```
dataset['flag_own_car'].value_counts()
```

| flag_own_car | count |
|---|---|
| N | 3262 |
| NO | 3148 |
| YES | 3100 |
| Y | 3084 |

dtype: int64

```
dataset['flag_own_car'] = dataset['flag_own_car'].astype(str).str.lower().str.s
dataset['flag_own_car'] = dataset['flag_own_car'].replace({'y': 1, 'yes': 1, 'n
dataset['flag_own_car'].fillna(0, inplace=True)
dataset['flag_own_car'] = dataset['flag_own_car'].astype(int)
```

```
/tmp/ipython-input-3989121408.py:2: FutureWarning: Downcasting behavior i
  dataset['flag_own_car'] = dataset['flag_own_car'].replace({'y': 1, 'yes
/tmp/ipython-input-3989121408.py:3: FutureWarning: A value is trying to b
The behavior will change in pandas 3.0. This inplace method will never wo

For example, when doing 'df[col].method(value, inplace=True)', try using


  dataset['flag_own_car'].fillna(0, inplace=True)
```

```
dataset['flag_own_realty'] = dataset['flag_own_realty'].astype(str).str.lower(
dataset['flag_own_realty'] = dataset['flag_own_realty'].replace({'y': 1, 'yes'
dataset['flag_own_realty'].fillna(0, inplace=True)
dataset['flag_own_realty'] = dataset['flag_own_realty'].astype(int)
```

```
/tmp/ipython-input-2180259029.py:2: FutureWarning: Downcasting behavior i
  dataset['flag_own_realty'] = dataset['flag_own_realty'].replace({'y': 1
/tmp/ipython-input-2180259029.py:3: FutureWarning: A value is trying to b
The behavior will change in pandas 3.0. This inplace method will never wo

For example, when doing 'df[col].method(value, inplace=True)', try using


  dataset['flag_own_realty'].fillna(0, inplace=True)
```

```
dataset['flag_own_realty'].value_counts()
```

| flag_own_realty | count |
|---|---|
| 0 | 9449 |
| 1 | 6301 |

dtype: int64

```
dataset.drop(['cnt_children'], axis=1, inplace=True)
```

```
dataset.drop(['amt_income_total'], axis=1, inplace=True)
```

```
dataset.drop('loan_amount', axis=1, inplace=True)
```

```
dataset['amt_credit'].value_counts()
```

| amt_credit | count |
|---|---|
| 300000 | 3192 |
| ₹200000 | 3168 |
| 500k | 3162 |
| 1000000 | 3159 |

dtype: int64

```
dataset['amt_credit'] = dataset['amt_credit'].astype(str).str.replace('[₹,]', '
dataset['amt_credit'] = pd.to_numeric(dataset['amt_credit'], errors='coerce')
dataset['amt_credit'].fillna(dataset['amt_credit'].median(), inplace=True)
```

```
/tmp/ipython-input-1715016252.py:3: FutureWarning: A value is trying to b
The behavior will change in pandas 3.0. This inplace method will never wo

For example, when doing 'df[col].method(value, inplace=True)', try using


  dataset['amt_credit'].fillna(dataset['amt_credit'].median(), inplace=Tr
```

```
def clean_credit(x):
    if pd.isna(x):
```

```
        return np.nan
    x = str(x).lower().replace('₹', '').replace(',', '').strip()
    if 'k' in x:
        return float(x.replace('k', '')) * 1000
    try:
        return float(x)
    except:
        return np.nan

dataset['amt_credit'] = dataset['amt_credit'].apply(clean_credit)
dataset['amt_credit'].fillna(dataset['amt_credit'].median(), inplace=True)
```

```
/tmp/ipython-input-107663778.py:13: FutureWarning: A value is trying to b
The behavior will change in pandas 3.0. This inplace method will never wo

For example, when doing 'df[col].method(value, inplace=True)', try using


  dataset['amt_credit'].fillna(dataset['amt_credit'].median(), inplace=Tr
```

```
dataset['amt_credit'].value_counts()
```

| amt_credit | count |
|---|---|
| 300000.0 | 9423 |
| 200000.0 | 3168 |
| 1000000.0 | 3159 |

dtype: int64

```
import pandas as pd

# Step 1: Define a cleaning function
def clean_amount(value):
    if pd.isna(value):
        return None
    value = str(value).strip().lower().replace('₹', '').replace(',', '').replac

    # Handle shorthand like 5k or 2.5m
    if 'k' in value:
        try:
            return float(value.replace('k', '')) * 1000
        except:
            return None
    elif 'm' in value:
        try:
            return float(value.replace('m', '')) * 1_000_000
        except:
            return None
    else:
        try:
```

```
            return float(value)
        except:
            return None

# Step 2: Apply to your column
dataset['amt_annuity'] = dataset['amt_annuity'].apply(clean_amount)

# Step 3: Fill missing values (optional — median is a safe choice)
dataset['amt_annuity'] = dataset['amt_annuity'].fillna(dataset['amt_annuity'].m
```

```
dataset['amt_annuity'] = dataset['amt_annuity'].astype(int)
```

```
dataset['amt_annuity'].value_counts()
```

| amt_annuity | count |
|---|---|
| 5000 | 6298 |
| 1000 | 3173 |
| 6000 | 3167 |
| 7000 | 3112 |

dtype: int64

```
# Step 1: Keep a backup before modifying
dataset['amt_annuity_backup'] = dataset['amt_annuity']

# Step 2: Convert all values to string
dataset['amt_annuity'] = dataset['amt_annuity'].astype(str)

# Step 3: Clean and normalize values safely
dataset['amt_annuity'] = (
    dataset['amt_annuity']
    .str.replace('₹', '', regex=False)
    .str.replace(',', '', regex=False)
    .str.replace('k', '000', regex=False)
    .astype(float)
)

# Step 4: Verify
dataset['amt_annuity'].value_counts()
```

| amt_annuity | count |
|---|---|
| 5000.0 | 6298 |
| 1000.0 | 3173 |
| 6000.0 | 3167 |
| 7000.0 | 3112 |

dtype: int64

```
cols = list(dataset.columns)
cols.remove('amt_credit')
cols.remove('amt_annuity')

cols.insert(6, 'amt_credit')
cols.insert(7, 'amt_annuity')

dataset = dataset[cols]
```

```
dataset['amt_credit'] = dataset['amt_credit'].astype(int)
```

```
dataset.drop('amt_goods_price', axis=1, inplace=True)
```

```
dataset.head(5)
```

| | customer_id | age | gender | marital_status | employment_type | annual_incom |
|---|---|---|---|---|---|---|
| 0 | CUST100000 | 56 | female | married | unemployed | 5000 |
| 1 | CUST100001 | 69 | male | married | salaried | 5000 |
| 2 | CUST100002 | 46 | other | single | salaried | 10000 |
| 3 | CUST100003 | 32 | female | widowed | unemployed | 12000 |
| 4 | CUST100004 | 60 | female | single | self-employed | 20000 |

```
dataset['name_income_type'].value_counts()
```

| name_income_type | count |
|---|---|
| Working | 2669 |
| Student | 2639 |
| Pensioner | 2617 |
| Workng | 2605 |
| Businessman | 2595 |

dtype: int64

```
dataset['name_income_type'] = dataset['name_income_type'].str.lower().replace(
```

```
dataset['name_income_type'].fillna(dataset['name_income_type'].mode()[0], inpla
```

```
/tmp/ipython-input-2645880089.py:1: FutureWarning: A value is trying to b
The behavior will change in pandas 3.0. This inplace method will never wo

For example, when doing 'df[col].method(value, inplace=True)', try using


  dataset['name_income_type'].fillna(dataset['name_income_type'].mode()[0
```

```
dataset.head(5)
```

| | customer_id | age | gender | marital_status | employment_type | annual_incom |
|---|---|---|---|---|---|---|
| 0 | CUST100000 | 56 | female | married | unemployed | 5000 |
| 1 | CUST100001 | 69 | male | married | salaried | 5000 |
| 2 | CUST100002 | 46 | other | single | salaried | 10000 |
| 3 | CUST100003 | 32 | female | widowed | unemployed | 12000 |
| 4 | CUST100004 | 60 | female | single | self-employed | 20000 |

```
dataset['name_education_type'] = dataset['name_education_type'].str.lower().str
dataset['name_education_type'].replace({
    'high': 'secondary',
    'incomplete higher': 'higher'
}, inplace=True)
```

```
dataset['name_education_type'].fillna(dataset['name_education_type'].mode()[0]
```

```
/tmp/ipython-input-4164629113.py:2: FutureWarning: A value is trying to b
The behavior will change in pandas 3.0. This inplace method will never wo

For example, when doing 'df[col].method(value, inplace=True)', try using


  dataset['name_education_type'].replace({
```

```
dataset['name_education_type'].value_counts()
```

| name_education_type | count |
|---|---|
| higher | 7871 |
| secondary | 5235 |
| academic degree | 2644 |

dtype: int64

```
dataset['name_family_status'] = dataset['name_family_status'].str.lower()
```

```
import numpy as np

# Create mask for missing values
mask = dataset['name_family_status'].isna()

# Randomly select existing values (uniform distribution)
random_choices = np.random.choice(
    dataset['name_family_status'].dropna().unique(),
    size=mask.sum(),
    replace=True
)

# Assign random choices to missing entries
dataset.loc[mask, 'name_family_status'] = random_choices
```

```
dataset['name_family_status'].value_counts()
```

| name_family_status | count |
|---|---|
| married | 4026 |
| separated | 4005 |
| widow | 3861 |
| single | 3858 |

dtype: int64

```
dataset.drop(['ext_source_1','ext_source_2','ext_source_3','amt_annuity_backup
```

```
dataset['name_housing_type'] = dataset['name_housing_type'].str.lower().str.re
```

```
import numpy as np

mask = dataset['name_housing_type'].isna()
value_probs = dataset['name_housing_type'].value_counts(normalize=True)

random_choices = np.random.choice(
    value_probs.index,
    size=mask.sum(),
    replace=True,
    p=value_probs.values
)

dataset.loc[mask, 'name_housing_type'] = random_choices
```

```
dataset['name_housing_type'].value_counts()
```

| name_housing_type | count |
|---|---|
| rent | 7837 |
| owned | 3980 |
| mortgage | 3933 |

dtype: int64

```
dataset['occupation_type'] = dataset['occupation_type'].str.lower()
```

```
dataset['occupation_type'].value_counts()
```

| occupation_type | count |
|---|---|
| laborer | 2288 |
| manager | 2286 |
| driver | 2283 |
| it staff | 2219 |
| drivr | 2209 |
| accountant | 2155 |

dtype: int64

```
import numpy as np

# Create mask for missing values
mask = dataset['occupation_type'].isna()

# Randomly fill missing with existing unique values (equal probability)
random_choices = np.random.choice(
    dataset['occupation_type'].dropna().unique(),
    size=mask.sum(),
    replace=True
)

# Assign the random values to missing rows
dataset.loc[mask, 'occupation_type'] = random_choices
```

```
dataset['occupation_type'].isna().sum()
```

```
np.int64(0)
```

```
dataset['occupation_type'] = dataset['occupation_type'].str.replace('drivr','dr
```

```
dataset.head(5)
```

| | customer_id | age | gender | marital_status | employment_type | annual_incom |
|---|---|---|---|---|---|---|
| 0 | CUST100000 | 56 | female | married | unemployed | 5000 |
| 1 | CUST100001 | 69 | male | married | salaried | 5000 |
| 2 | CUST100002 | 46 | other | single | salaried | 10000 |
| 3 | CUST100003 | 32 | female | widowed | unemployed | 12000 |
| 4 | CUST100004 | 60 | female | single | self-employed | 20000 |

```
import numpy as np

# Identify NaN values
mask = dataset['region_rating_client'].isna()

# Randomly assign 1, 2, or 3 to those NaNs
dataset.loc[mask, 'region_rating_client'] = np.random.choice([1, 2, 3], size=ma
```

```
dataset['weekday_appr_process_start'].value_counts()
```

| weekday_appr_process_start | count |
|---|---|
| Tuesday | 2749 |
| Friday | 2645 |
| Thur | 2612 |
| Mon | 2553 |
| Wed | 2526 |

dtype: int64

```
# Create mapping dictionary
weekday_mapping = {
    'mon': 'monday',
    'tue': 'tuesday',
    'tues': 'tuesday',
    'wed': 'wednesday',
    'thu': 'thursday',
    'thurs': 'thursday',
    'fri': 'friday',
    'sat': 'saturday',
    'sun': 'sunday',
    'thur':'thursday'
```

```
}
```

```
# Convert to lowercase and strip spaces first
dataset['weekday_appr_process_start'] = dataset['weekday_appr_process_start'].s

# Replace abbreviations with full names
dataset['weekday_appr_process_start'] = dataset['weekday_appr_process_start'].r
```

```
dataset['weekday_appr_process_start'].value_counts()
```

| weekday_appr_process_start | count |
|---|---|
| tuesday | 2749 |
| friday | 2645 |
| thursday | 2612 |
| monday | 2553 |
| wednesday | 2526 |

dtype: int64

```
import numpy as np

mask = dataset['weekday_appr_process_start'].isna()

dataset.loc[mask, 'weekday_appr_process_start'] = np.random.choice(
    ['monday', 'tuesday', 'wednesday', 'thursday', 'friday'],
    size=mask.sum(),
    replace=True
)
```

```
import numpy as np

mask = dataset['hour_appr_process_start'].isna()

dataset.loc[mask, 'hour_appr_process_start'] = np.random.randint(0, 24, size=ma
```

```
dataset['hour_appr_process_start'].dtype
```

```
dtype('O')
```

```
dataset['hour_appr_process_start'] = dataset['hour_appr_process_start'].str.rep
```

```
import numpy as np

# Step 1: Fill missing values with random valid hours (0–23)
mask = dataset['hour_appr_process_start'].isna()
dataset.loc[mask, 'hour_appr_process_start'] = np.random.randint(0, 24, size=ma

# Step 2: Convert column to integer safely
dataset['hour_appr_process_start'] = dataset['hour_appr_process_start'].astype(
```

```
def categorize_hour(hour):
    if 5 <= hour < 12:
        return 'morning'
    elif 12 <= hour < 17:
        return 'afternoon'
    elif 17 <= hour < 21:
        return 'evening'
    else:
        return 'night'

dataset['hour_category'] = dataset['hour_appr_process_start'].apply(categorize_
```

```
dataset['organization_type'] = dataset['organization_type'].str.lower().str.rep
```

```
dataset[['date_of_birth','age']].head(10)
```

| | date_of_birth | age |
|---|---|---|
| 0 | 1995-02-19 | 30 |
| 1 | 1992-09-30 | 33 |
| 2 | 1971-08-02 | 54 |
| 3 | 1990-01-09 | 35 |
| 4 | 1959-04-11 | 66 |
| 5 | 1974-10-09 | 51 |
| 6 | 1971-07-26 | 54 |
| 7 | 1993-04-07 | 32 |
| 8 | 1968-08-16 | 57 |
| 9 | 1984-03-07 | 41 |

```
import pandas as pd
from datetime import datetime, timedelta

# Step 1: Ensure both columns are numeric
dataset['days_birth'] = pd.to_numeric(dataset['days_birth'], errors='coerce')
dataset['age'] = pd.to_numeric(dataset['age'], errors='coerce')

# Step 2: Get today's date
today = pd.Timestamp.now().normalize()  # normalized to midnight

# Step 3: Convert days_birth into date_of_birth
# (days_birth is negative, so we add it to today's date)
dataset['date_of_birth'] = today + pd.to_timedelta(dataset['days_birth'], unit=

# Step 4: Optional — sanity check (verify the conversion matches 'age')
dataset['calculated_age'] = ((today - dataset['date_of_birth']).dt.days / 365)

# Step 5: Compare a few values
print(dataset[[ 'age',  'date_of_birth']].head())
```

```
dataset[['date_of_birth','age']].head(5)
```

| | date_of_birth | age |
|---|---|---|
| 0 | 1995-02-19 | 30 |
| 1 | 1992-09-30 | 33 |
| 2 | 1971-08-02 | 54 |
| 3 | 1990-01-09 | 35 |
| 4 | 1959-04-11 | 66 |

```
dataset.drop('age', axis=1, inplace=True)
```

```
# Get the list of columns
cols = list(dataset.columns)

# Find the positions of customer_id and gender
pos_customer = cols.index('customer_id')
pos_gender = cols.index('gender')

# Remove the columns if they already exist elsewhere
cols.remove('date_of_birth')
cols.remove('calculated_age')

# Insert 'days_birth' and 'calculated_age' between them
cols.insert(pos_gender, 'calculated_age')
cols.insert(pos_gender, 'date_of_birth')

# Reorder the DataFrame
dataset = dataset[cols]
```